Cutting Planes Are Costates: Neural Stochastic Dual Dynamic Programming for the Analytic Climate Economy

Herman Vestvik Thieme

November 5, 2026

Draft

Abstract

Stochastic dual dynamic programming (SDDP) approximates the value function of a multistage problem by cutting planes, and neural SDDP learns those cutting planes across problem instances. I apply both to the Analytic Climate Economy (ACE), an integrated assessment model with DICE functional forms and a closed-form solution, so that every result can be checked exactly. With business-as-usual emissions on an exogenous path, each period of ACE is a small convex program, and the gradient of its value, obtained from the multipliers on the cuts, is the costate recursion of the model's equilibrium conditions: the slopes of the cuts are ACE's costates, and the social cost of carbon (SCC) follows from them. Because ACE's value function is affine, SDDP is exact after two iterations; it reproduces the closed-form SCC to within 0.002% across a wide range of time preferences, damages and carbon-cycle parameters, in under three seconds per parameter draw. A cut network trained on 128 SDDP solutions predicts the cuts for new parameter values; one forward pass with these cuts, without any SDDP iterations, gives the SCC with a median error of 0.2% (at most 2.5%), 8 to 19 times more accurate than a pseudo-state deep equilibrium net on the same test draws. Warm-starting SDDP with the predicted cuts makes its first policy nearly optimal but does not reduce the number of iterations, because SDDP from scratch already needs only one backward pass. I discuss what these results imply for climate-economy models without closed forms, where curved value functions would let neural SDDP pay off, and where non-convexities such as tipping points would break SDDP's guarantees.

1 Introduction

Integrated assessment models (IAMs) link a model of the economy to a model of the climate and ask what a tonne of carbon dioxide costs society. Their answer, the social cost of carbon (SCC), is a shadow price: the marginal value of carbon in the atmosphere, relative to the marginal value of consumption. Computing it requires solving a dynamic optimisation problem over centuries, and the moment such a problem has more than a handful of state variables, or uncertainty, or tipping points, conventional grid-based dynamic programming runs into the curse of dimensionality. A growing literature therefore turns to methods from machine learning: deep equilibrium nets, which train a network to satisfy a model's equilibrium conditions (Azinovic et al. 2022; Maliar et al. 2021; Folini et al. 2024), their extension to uncertain parameters (Friedl et al. 2023), and reinforcement learning.

Operations research has a different and much older answer to the same problem. Stochastic dual dynamic programming (SDDP) (Pereira and Pinto 1991) solves multistage stochastic programs,

originally for hydrothermal power systems, by approximating each stage's value function from above with cutting planes (Kelley 1960; Benders 1962). It alternates forward passes, which simulate the policy implied by the current cuts, with backward passes, which add a cut at each visited state using the value and the gradient of the stage problem. It needs convexity, but where convexity holds it comes with bounds on the optimum and convergence guarantees (Shapiro 2011; Girardeau et al. 2015). Neural SDDP (Dai et al. 2022) adds a network that maps a problem instance to its cuts, trained on earlier solutions, so that new instances can be solved faster or without iterations at all.

This paper applies both methods to the Analytic Climate Economy (ACE) of Traeger (2023), in the DICE calibrations used by Traeger (2021). ACE keeps the DICE production function, carbon cycle and temperature dynamics, but transforms capital and temperature so that the value function is affine and the model has a closed-form solution. That makes it a demanding benchmark: every number a method produces can be compared with the exact one.

The contributions are the following.

1. **ACE as a sequence of convex programs.** With business-as-usual emissions on an exogenous path, which is the assumption behind ACE's closed form, each period's problem is convex as long as the cuts value capital positively (Proposition 1). The gradient of the stage value, obtained from the multipliers on the cuts by the envelope theorem, is exactly the costate recursion of the model's equilibrium conditions (Proposition 2). The slopes of the cuts are therefore ACE's costates, and the SCC follows from the slopes of the next period's cut. This is a third route to the same shadow prices, next to the residuals of the equilibrium conditions used by deep equilibrium nets and the gradient of a learned value function used by actor-critic methods.

2. **SDDP is exact in two iterations.** Because the value function is affine, a cut computed at any state is exact, the closed-form cut is a fixed point of the backward pass (Proposition 3), and SDDP from scratch converges after one backward pass (Corollary 1). Across 256 test draws of the time preference, the damage coefficient and the speed of atmosphere–ocean carbon exchange, it reproduces the closed-form SCC to within 0.002%, in under three seconds per draw.

3. **Neural SDDP predicts the cuts across parameter values.** A cut network trained on the SDDP solutions of 128 parameter draws gives, with a single forward pass and no SDDP iterations, a median SCC error of 0.24% (DICE 2016) and 0.21% (DICE 2023), at most 2.5%, and a welfare loss below 0.0006% on every test draw. On the same draws, this is 8 to 19 times more accurate than a pseudo-state deep equilibrium net trained over the same parameter box.

4. **Warm starts do not help in ACE, and I explain why.** The predicted cuts make the first forward pass nearly optimal, within 10^{-9} of the optimum against 12% below it from scratch. But certifying convergence requires a backward pass either way, and from scratch SDDP needs only one. The speed-ups of neural SDDP require value functions that need many cuts.

5. **Uncertainty and the horizon.** With i.i.d. productivity shocks, SDDP proper (averaging cuts over shocks, sampling them forward) recovers the same costates and policy, as ACE predicts. The horizon after which the value is set to zero has to be long at low discount rates, because carbon's shadow price converges only at the rate β (Remark 1).

The paper is organised as follows. Section 2 reviews related work. Section 3 presents the model and its closed form. Section 4 formulates the stage problems, derives the cuts and describes SDDP and neural SDDP. Section 5 describes the experiments, Section 6 the results, and Section 7 discusses what they imply for models without closed forms.

2 Related Work

SDDP. SDDP was introduced by Pereira and Pinto (1991) for the planning of hydrothermal power systems, building on the cutting-plane method of Kelley (1960) and the decomposition of Benders (1962). Shapiro (2011) analyses its statistical properties and stopping rules, and Girardeau et al. (2015) prove convergence for multistage stochastic convex programs, the setting of this paper; Dowson and Kapelevich (2021) provide a widely used implementation. SDDP relies on convexity of the value functions in the state and on stagewise-independent uncertainty; both hold in ACE with exogenous business-as-usual emissions and i.i.d. shocks.

Learning value functions for multistage problems. Dai et al. (2022) introduce neural SDDP (ν-SDDP), in which a network maps a problem instance to a piecewise-linear value function in a learned low-dimensional projection of the state, and use it to warm-start or replace SDDP on new instances. Related approaches learn the gradients of the value function (Lee et al. 2023), represent it with input-convex neural networks (Bae et al. 2023), or generate cuts sequentially with a transformer (Kim et al. 2023). These papers evaluate their methods on inventory, production and energy planning problems against SDDP solutions; here the benchmark is a closed form.

Solving IAMs. Analytic IAMs include Golosov et al. (2014) and ACE (Traeger 2023). Numerical approaches include dynamic programming with risk (Cai and Lontzek 2019) and tipping points (Lemoine and Traeger 2014), and deep learning: deep equilibrium nets (Azinovic et al. 2022), applied to climate economics by Folini et al. (2024), and deep uncertainty quantification, which treats uncertain parameters as pseudo-states (Friedl et al. 2023). The companion revision of the thesis behind this repository (Thieme 2025) corrects the equilibrium conditions of a deep equilibrium net for ACE; its pseudo-state extension is the comparison in Section 6. To my knowledge, SDDP and neural SDDP have not been applied to an IAM with a closed-form benchmark.

3 The Analytic Climate Economy

I use ACE (Traeger 2023) with the DICE production function, carbon cycle and two-layer temperature model, in the calibrations of DICE 2016 (Nordhaus 2018) and DICE 2023 (Barrage and Nordhaus 2023), following Traeger (2021). Time is discrete, with periods of $\Delta = 5$ years.

States, decisions and reward. The state is $\boldsymbol{s}_t = (k_t, \boldsymbol{M}_t, \boldsymbol{\tau}_t) \in \mathbb{R}^6$, where $k_t = \log K_t$ is log capital, $\boldsymbol{M}_t \in \mathbb{R}^3$ are the carbon stocks in the atmosphere, upper ocean and deep ocean (GtC), and $\boldsymbol{\tau}_t \in \mathbb{R}^2$ are transformed atmospheric and ocean temperatures, $\tau_{i,t} = \exp(\xi_1 T_{i,t})$. In each period the planner chooses the consumption rate $x_t \in (0, 1)$ and the abatement rate $\mu_t \in [0, 1]$. Log output net of abatement costs and damages is

$$g_t(\boldsymbol{s}_t, \mu_t) = \log A_t + (1 - \kappa) \log N_t + \kappa k_t + \log\left(1 - \theta_{1,t}\mu_t^{\theta_2}\right) - \xi_0 \tau_{1,t} + \xi_0, \tag{1}$$

with exogenous productivity A_t, labour N_t and abatement-cost coefficients $\theta_{1,t}$ from DICE, and $\theta_2 = 2.6$. The reward is log consumption, $u_t = \log x_t + g_t(\boldsymbol{s}_t, \mu_t)$, and the planner maximises $\sum_t \beta^t u_t$ with $\beta = (1+\rho)^{-\Delta}$.

Laws of motion. Capital, carbon and temperatures evolve as

$$k_{t+1} = g_t(\boldsymbol{s}_t, \mu_t) + \log(1 - x_t) + c_k, \tag{2}$$

$$\boldsymbol{M}_{t+1} = \boldsymbol{\Phi}\boldsymbol{M}_t + \boldsymbol{e}_1 \zeta \left[(1-\mu_t)E_t^{\text{BAU}} + E_t^{\text{land}}\right], \tag{3}$$

$$\boldsymbol{\tau}_{t+1} = \boldsymbol{\sigma}\boldsymbol{\tau}_t + \boldsymbol{e}_1 \sigma^{\text{forc}} \frac{M_{1,t}}{M_{\text{pre}}}, \tag{4}$$

where $c_k = \log[(1 + g_k\Delta)/(\delta_k + g_k)]$, emissions are measured in $GtCO_2$ per year and $\zeta = \Delta/3.664$ converts them into GtC per period. The carbon transition matrix $\boldsymbol{\Phi}$ is column-stochastic: it moves carbon between reservoirs and conserves it.

Business-as-usual emissions. In DICE, business-as-usual emissions are proportional to gross output, $E_t^{\text{BAU}} = \sigma_t Y_t^{\text{gross}}$, and so depend on capital. ACE's closed form requires the planner to take them as given. As in the actor-critic paper of this series, I fix E_t^{BAU} to an exogenous reference path: carbon intensity times gross output along the optimal path at baseline parameters. The problem solved by SDDP and the problem with the closed-form solution are then the same.

Uncertain parameters. Three parameters vary across instances, over the box used for deep uncertainty quantification in this series: the pure rate of time preference $\rho \in [0.5\%, 3.5\%]$ per year, the damage coefficient $\xi_0 \in [0.011, 0.044]$ (half to twice its baseline), and a multiplier $\omega \in [0.75, 1.25]$ on the exchange of carbon between the atmosphere and the upper ocean, which scales the two corresponding entries of $\boldsymbol{\Phi}$ and keeps it column-stochastic. I write $\boldsymbol{p} = (\rho, \xi_0, \omega)$ for an instance.

Closed form. The value function is affine, $V_t(\boldsymbol{s}) = \alpha_t + \boldsymbol{b}^\top \boldsymbol{s}$, with slopes that are constant over time:

$$b_k = \frac{\kappa}{1 - \beta\kappa}, \tag{5}$$

$$\boldsymbol{b}_\tau = -\xi_0(1 + \beta b_k)\left(\boldsymbol{I} - \beta\boldsymbol{\sigma}^\top\right)^{-1}\boldsymbol{e}_1, \tag{6}$$

$$\boldsymbol{b}_M = \beta \frac{\sigma^{\text{forc}}}{M_{\text{pre}}} b_{\tau_1}\left(\boldsymbol{I} - \beta\boldsymbol{\Phi}^\top\right)^{-1}\boldsymbol{e}_1. \tag{7}$$

In the notation of the deep equilibrium nets in this series, the costates are $\boldsymbol{\lambda} = \beta\boldsymbol{b}$. The optimal consumption rate is $x^* = 1/(1 + \beta b_k) = 1 - \beta\kappa$, and the optimal abatement rate equates the marginal product of emissions to the carbon price,

$$\frac{\theta_{1,t}\theta_2 \mu_t^{\theta_2 - 1}}{E_t^{\text{BAU}}\left(1 - \theta_{1,t}\mu_t^{\theta_2}\right)} = p^C \equiv -\frac{\zeta\beta b_{M_1}}{1 + \beta b_k}, \tag{8}$$

with $\mu_t^* = 1$ where the solution would exceed one. The SCC in USD per tCO_2 is

$$SCC_t = 1000\, Y_t^{\text{net}}\, p^C = -1000\, Y_t^{\text{net}}\, \frac{\zeta\beta b_{M_1}}{1 + \beta b_k}, \tag{9}$$

where $Y_t^{\text{net}} = \exp(g_t)$ is net output in trillion USD.

4 Method

4.1 Stage problems

Let $\mathcal{C}_{t+1}$ be a set of cuts $(\alpha_j, \boldsymbol{b}_j)$ with $V_{t+1}(\boldsymbol{s}) \le \alpha_j + \boldsymbol{b}_j^\top \boldsymbol{s}$. Writing $\boldsymbol{f}_t(\boldsymbol{s}, x, \mu)$ for the laws of motion (2)–(4), the stage problem at state $\boldsymbol{s}$ is

$$\begin{aligned} \hat{V}_t(\boldsymbol{s}) = \max_{x,\mu,\theta} \quad & \log x + g_t(\boldsymbol{s}, \mu) + \beta\theta \\ \text{s.t.} \quad & \theta \le \alpha_j + \boldsymbol{b}_j^\top \boldsymbol{f}_t(\boldsymbol{s}, x, \mu), \qquad j \in \mathcal{C}_{t+1}, \\ & \underline{x} \le x \le \overline{x}, \quad 0 \le \mu \le 1. \end{aligned} \tag{P_t}$$

Proposition 1 (Convexity)**.** *If every cut has $b_{j,k} \ge 0$, problem (P_t) is a convex program.*

Proof. Since $\theta_2 > 1$, μ^{θ_2} is convex on $[0,1]$, so $\log(1 - \theta_{1,t}\mu^{\theta_2})$ is concave, and with $\log x$ and $\log(1-x)$ the objective and next capital (2) are concave in (x, μ). Next carbon (3) is affine in μ, and next temperatures (4) do not depend on the decisions. The right-hand side of each cut constraint is therefore concave when $b_{j,k} \ge 0$, and the feasible set $\{\theta \le \text{concave}\}$ is convex. □

The condition $b_{j,k} \ge 0$ says that capital is valuable; it holds for every cut SDDP computes, and I enforce it for predicted cuts. I implement (P_t) once in CVXPY (Diamond and Boyd 2016) in disciplined parametrised form (Agrawal et al. 2019), introducing auxiliary variables $w \le \log(1 - \theta_{1,t}\mu^{\theta_2})$ and $z \le w + \log(1-x)$ so that the cut slopes on capital multiply a variable rather than an expression with parameters. The problem is then re-solved with new parameters for every stage, state and cut set, with the interior-point solver Clarabel (Goulart and Chen 2024).

4.2 Cuts from duals

Proposition 2 (Cut gradient)**.** *Let $\pi_j \ge 0$ be the multipliers of the cut constraints at a solution of (P_t); they satisfy $\sum_j \pi_j = \beta$. Define $w_k = \sum_j \pi_j b_{j,k}$, $\boldsymbol{w}_M = \sum_j \pi_j \boldsymbol{b}_{j,M}$ and $\boldsymbol{w}_\tau = \sum_j \pi_j \boldsymbol{b}_{j,\tau}$. Then*

$$\frac{\partial \hat{V}_t}{\partial k} = \kappa\,(1 + w_k), \tag{10}$$

$$\frac{\partial \hat{V}_t}{\partial \boldsymbol{M}} = \boldsymbol{\Phi}^\top \boldsymbol{w}_M + \frac{\sigma^{forc}}{M_{pre}}\, w_{\tau_1}\, \boldsymbol{e}_1, \tag{11}$$

$$\frac{\partial \hat{V}_t}{\partial \boldsymbol{\tau}} = \boldsymbol{\sigma}^\top \boldsymbol{w}_\tau - \xi_0\,(1 + w_k)\,\boldsymbol{e}_1. \tag{12}$$

Proof. Stationarity of the Lagrangian in θ gives $\sum_j \pi_j = \beta$. The state enters (P_t) only through g_t and $\boldsymbol{f}_t$, so by the envelope theorem $\nabla_{\boldsymbol{s}} \hat{V}_t = \nabla_{\boldsymbol{s}} g_t + \sum_j \pi_j \boldsymbol{J}_t^\top \boldsymbol{b}_j$, where $\boldsymbol{J}_t$ is the Jacobian of $\boldsymbol{f}_t$ with respect to the state: $\partial k_{t+1}/\partial k_t = \kappa$, $\partial k_{t+1}/\partial \tau_{1,t} = -\xi_0$, $\partial \boldsymbol{M}_{t+1}/\partial \boldsymbol{M}_t = \boldsymbol{\Phi}$, $\partial \tau_{1,t+1}/\partial M_{1,t} = \sigma^{\mathrm{forc}}/M_{\mathrm{pre}}$ and $\partial \boldsymbol{\tau}_{t+1}/\partial \boldsymbol{\tau}_t = \boldsymbol{\sigma}$. Collecting terms gives the result. □

With $\boldsymbol{\lambda}_t = \sum_j \pi_j \boldsymbol{b}_j = (w_k, \boldsymbol{w}_M, \boldsymbol{w}_\tau)$, these are the costate equations of ACE: $\lambda_{k,t-1} = \beta\kappa(1 + \lambda_{k,t})$ and its counterparts for carbon and temperature, which deep equilibrium nets enforce as residuals. A backward pass of SDDP evaluates them from the duals of convex programs. The new cut for V_t at the visited state $\boldsymbol{s}$ is $(\hat{V}_t(\boldsymbol{s}) - \nabla \hat{V}_t^\top \boldsymbol{s},\ \nabla \hat{V}_t)$, and the SCC in period t follows from (9) with the multiplier-weighted slopes $\boldsymbol{\lambda}_t/\beta$ of the next period's cuts.

Algorithm 1 SDDP for ACE

$\mathcal{C}_t \leftarrow \{(\bar{V}, \mathbf{0})\}$ (provisional) for $t < T$; $\mathcal{C}_T \leftarrow \{(0, \mathbf{0})\}$; optionally replace $\mathcal{C}_t$ by predicted cuts for $1 \le t < T$
for $i = 1, 2, \ldots$ **do**
  Forward: $\boldsymbol{s}_0$ given; for $t = 0, \ldots, T-1$: solve (P_t) at $\boldsymbol{s}_t$ with $\mathcal{C}_{t+1}$, $\boldsymbol{s}_{t+1} \leftarrow \boldsymbol{f}_t(\boldsymbol{s}_t, x_t, \mu_t)$
  $\underline{V} \leftarrow \sum_t \beta^t u_t$; $\bar{V}_0 \leftarrow \min_{j \in \mathcal{C}_0} \alpha_j + \boldsymbol{b}_j^\top \boldsymbol{s}_0$
  if $(\bar{V}_0 - \underline{V})/|\underline{V}| < \epsilon$ **then stop**
  end if
  Backward: for $t = T-1, \ldots, 0$: solve (P_t) at $\boldsymbol{s}_t$ with $\mathcal{C}_{t+1}$; add the cut $(\hat{V}_t - \nabla \hat{V}_t^\top \boldsymbol{s}_t, \nabla \hat{V}_t)$ to $\mathcal{C}_t$, replacing provisional cuts
end for

Proposition 3 (The closed form is a fixed point)**.** *If $\mathcal{C}_{t+1}$ consists of the single cut $(\alpha, \boldsymbol{b})$ with $\boldsymbol{b}$ from* (5)–(7) *and any α, then* (P_t) *is solved by $x = 1 - \beta\kappa$ and the abatement rate μ_t^* of* (8)*, at every state, and its gradient is $\boldsymbol{b}$.*

Proof. With one cut, $\pi = \beta$ and $\boldsymbol{\lambda}_t = \beta \boldsymbol{b}$. The first-order condition for x is $1/x = \beta b_k/(1-x)$, so $x = 1/(1+\beta b_k) = 1 - \beta\kappa$; the condition for μ is (8). Neither depends on the state. By Proposition 2, $\partial \hat{V}_t/\partial k = \kappa(1+\beta b_k) = b_k$ by (5); $\partial \hat{V}_t/\partial \boldsymbol{\tau} = \beta \boldsymbol{\sigma}^\top \boldsymbol{b}_\tau - \xi_0(1+\beta b_k)\boldsymbol{e}_1 = \boldsymbol{b}_\tau$ by (6); and $\partial \hat{V}_t/\partial \boldsymbol{M} = \beta \boldsymbol{\Phi}^\top \boldsymbol{b}_M + \beta(\sigma^{\mathrm{forc}}/M_{\mathrm{pre}}) b_{\tau_1} \boldsymbol{e}_1 = \boldsymbol{b}_M$ by (7). □

Corollary 1 (Two iterations)**.** *In the deterministic problem truncated after T stages with $V_T = 0$, and with exact stage solutions, SDDP started from any valid upper bound converges after two forward passes and one backward pass.*

Proof. By the argument of Proposition 3, if V_{t+1} is affine, the optimal decisions at stage t do not depend on the state, and since the reward and the laws of motion are affine in the state for given decisions, V_t is affine too. Starting from $V_T = 0$, every V_t is affine, so the first backward pass, which proceeds from the last stage to the first, computes at each stage a cut that equals V_t everywhere, whatever state it was computed at. The second forward pass then follows the optimal policy, and the cut at stage 0 equals the value of that policy. □

Remark 1 (Truncation)*.* The finite-horizon slopes follow the linear recursion of Proposition 2 backwards from zero. The slope on capital converges at the rate $\beta\kappa$ per period and the slopes on temperature at a rate below β, but the slopes on carbon converge only at the rate β: $\boldsymbol{\Phi}$ is column-stochastic, so $\boldsymbol{\Phi}^\top$ has the eigenvalue one, the mode of carbon that is never removed from the system. The error in the SCC in period t therefore decays like β^{T-t}. At $\rho = 0.5\%$ a year, $\beta = 0.975$ per period and $\beta^{150-58} \approx 0.10$, so a horizon of 150 periods is too short for an evaluation window of 58 periods; I use $T = 450$.

4.3 SDDP

Algorithm 1 summarises the method. Each stage starts with a provisional cut $V_t \le \bar{V}$; this cut, and any warm-start cuts, are replaced by the first cut a backward pass computes, so that only computed cuts determine the upper bound and later decisions. At most 16 cuts are kept per stage. The forward pass gives a lower bound $\underline{V}$, the value of a feasible policy; the stage-0 cuts evaluated at $\boldsymbol{s}_0$ give an upper bound $\bar{V}_0$; the algorithm stops when $(\bar{V}_0 - \underline{V})/|\underline{V}| < 10^{-7}$.

Shocks. To test the stochastic machinery, log productivity receives an i.i.d. shock $\varepsilon_t \sim \mathcal{N}(0, 0.03^2)$, observed before the decisions of period t. The backward pass then averages the values and gradients of (P_t) over five Gauss–Hermite nodes, the exact expectation for a polynomial of degree up to nine in the shock, and the forward pass samples the shocks. The shock enters g_t additively, so it shifts the value function but not its slopes: the optimal policy is the same as without it.

4.4 Neural SDDP

The cut network $h_\omega(\boldsymbol{p}, t)$ maps an instance and a stage to a cut $(\hat{\alpha}_t, \hat{\boldsymbol{b}}_t)$. The slopes have fixed signs (capital is valuable, carbon and temperature are harmful) and span several orders of magnitude, from about 0.4 for capital to 10^{-8} for deep-ocean carbon, so the network predicts $\log|\boldsymbol{b}|$ and the intercept, each standardised over the training set. The error is then relative, the same for a low and a high SCC. The inputs are the parameters, scaled to $[-1, 1]$, and the stage. Unlike Dai et al. (2022), who predict several cuts in a learned low-dimensional space, I predict one cut per stage in the original state: ACE's value function is affine, and its state has only six dimensions.

The network is trained by supervised learning on the final cuts of SDDP solutions for training draws of $\boldsymbol{p}$, with a mean squared error. For a new instance it is used in two ways:

- **No iterations:** one forward pass of Algorithm 1 with the predicted cuts gives a policy, the SCC along the path and its value;
- **Warm start:** the predicted cuts replace the provisional cuts, and SDDP runs to convergence.

5 Experimental Setup

Instances. For each calibration, DICE 2016 and DICE 2023, the training set consists of 128 instances and the test set of 256, drawn as scrambled Sobol sequences over the parameter box of Section 3 with different seeds. The test draws are the same points as those used to evaluate the pseudo-state deep equilibrium net in this series, so the two methods are compared on identical instances.

SDDP. The horizon is $T = 450$ periods; the evaluation window is the first 58 periods, up to about 2300. The consumption rate is bounded to $[0.2, 0.98]$, which never binds at the optimum, the provisional bound is $\bar{V} = 1000$, and the tolerance is $\epsilon = 10^{-7}$. A stage problem takes one to two milliseconds including Python overhead. Instances are solved in parallel in four processes on four CPU cores; the reported times per instance are the times within one process.

Cut network. The network has three hidden layers of 128 units with swish activations and is trained with Adam (Kingma and Ba 2017) for 2,000 epochs, with batches of 1,024 and a learning rate decaying from 10^{-3} to 10^{-5} on a cosine schedule. Its training set has 57,600 rows: 450 stages for each of the 128 training instances. One seed is used per calibration.

Pseudo-state deep equilibrium net. The comparison is the deep equilibrium net of the companion module on deep uncertainty quantification: a network with two hidden layers of 512 units that takes the state and the three parameters as inputs and is trained for 6,000 iterations on the

Table 1: Accuracy on 256 test instances: median over instances, with the maximum in parentheses. Errors are against the closed form over the periods up to about 2300.

	SDDP	Neural SDDP, no iterations	Neural SDDP, warm-started	Pseudo-state DEQN
DICE 2016				
SCC, mean relative error (%)	0.0007 (0.002)	0.24 (2.1)	0.0007 (0.002)	1.8 (19)
SCC in the first period, relative error (%)	0.00001 (0.0008)	0.37 (3.3)	0.00001 (0.0008)	2.0 (21)
Mean $\lvert\mu - \mu^*\rvert$	0.00001 (0.00003)	0.0003 (0.003)	0.00001 (0.00003)	0.003 (0.028)
Max $\lvert x - x^*\rvert$	0.00002 (0.00005)	0.00003 (0.00007)	0.00002 (0.00005)	0.006 (0.051)
Welfare loss (%)	$< 10^{-7}$	< 0.0002	$< 10^{-7}$	–
DICE 2023				
SCC, mean relative error (%)	0.0007 (0.002)	0.21 (2.5)	0.0007 (0.002)	4.0 (35)
SCC in the first period, relative error (%)	0.00001 (0.0008)	0.26 (4.2)	0.00001 (0.0008)	3.6 (33)
Mean $\lvert\mu - \mu^*\rvert$	0.00001 (0.00002)	0.0003 (0.004)	0.00001 (0.00002)	0.011 (0.059)
Max $\lvert x - x^*\rvert$	0.00002 (0.00003)	0.00003 (0.0001)	0.00002 (0.00004)	0.010 (0.047)
Welfare loss (%)	$< 10^{-7}$	< 0.0006	$< 10^{-7}$	–

Table 2: Cost per test instance (median, one process) and training cost. Times for the two calibrations differ by less than 10%; the table shows DICE 2016.

	Iterations	Stage problems	Seconds
SDDP from scratch	2	1,350	2.5
Neural SDDP, no iterations	–	450	0.9
Neural SDDP, warm-started	2	1,350	2.6
Training (wall time on four cores)			
SDDP solutions of 128 training instances			90
Cut network			672
Pseudo-state DEQN, for comparison			3,170

corrected equilibrium conditions of ACE over the same parameter box. Its benchmark lets business-as-usual emissions move with capital in the laws of motion, while the planner takes them as given; the two benchmarks therefore differ slightly, but relative errors against each method's own closed form are comparable.

Metrics. For every test instance I report, over the evaluation window, the mean relative error of the SCC along the path, the relative error of the SCC in the first period, the mean and maximum absolute errors of the abatement rate, the maximum absolute error of the consumption rate, and the consumption-equivalent welfare loss over the whole horizon, $\exp[(V^* - V)(1-\beta)] - 1$. Medians and maxima are taken over the 256 test instances.

6 Results

Table 1 summarises the accuracy of each method on the 256 test instances, and Table 2 its cost.

Table 3: Mean relative SCC error over the evaluation window (%) for different horizons, DICE 2016.

Horizon T (periods)	100	150	250	350	450
$\rho = 0.5\%$, $\xi_0 \times 2$, $\omega = 0.75$	15.0	4.2	0.32	0.024	0.002
Baseline	0.027	0.0006	0.0005	0.0006	0.0006
$\rho = 3.5\%$, $\xi_0 \times 0.5$, $\omega = 1.25$	0.007	0.0007	0.0004	0.0006	0.0006

6.1 SDDP reproduces the closed form

SDDP converges after two iterations on every instance, as Corollary 1 predicts, with 1,350 stage problems: two forward passes and one backward pass over 450 stages. Its SCC matches the closed form to within 0.002% on every test instance, its abatement rate to within 3×10^{-4} and its consumption rate to within 5×10^{-5} in every period; these residual errors are the solver's tolerance. The cut slopes agree with (5)–(7): over the evaluation window, the largest relative deviation of any slope is 10^{-15} at baseline parameters and 8×10^{-4} at the lowest discount rate and highest damages, where the truncation of Remark 1 remains. The baseline SCC in 2050 is \$36.98 per tCO_2 for DICE 2016 and \$41.99 for DICE 2023, equal to the closed form to the cent.

Table 3 shows why the horizon has to be long. With the lowest discount rate and the highest damages, a horizon of 150 periods leaves a mean SCC error of 4.2% over the evaluation window, and 100 periods an error of 15%; 450 periods bring it to 0.002%. At the baseline and at high discount rates, 150 periods already suffice. The errors decay roughly like β^{T-58}, as Remark 1 predicts.

6.2 Neural SDDP without iterations

A single forward pass with the predicted cuts gives the SCC with a median error of 0.24% for DICE 2016 and 0.21% for DICE 2023, and at most 2.1% and 2.5%; the abatement rate is within 0.0003 of the optimum in the median, and the welfare loss is below 0.0006% on every test instance (Table 1). The predicted SCC in the first period tracks the closed form across its full range, from about \$5 to \$140 per tCO_2 (Figure 2), and the predicted paths of abatement and the SCC follow the closed form to 2200, including the period in which abatement reaches 100% (Figure 3). As for the deep equilibrium net, the largest errors occur at low time preference: the median SCC error is 0.5% for $\rho \leq 1\%$ and 0.2% elsewhere (Figure 1). A forward pass costs a third of an SDDP solution: 450 stage problems instead of 1,350, and 0.9 seconds instead of 2.5. Predicting the cuts takes 7 milliseconds.

6.3 Comparison with a pseudo-state deep equilibrium net

On the same test instances, the pseudo-state deep equilibrium net has a median SCC error of 1.8% for DICE 2016 and 4.0% for DICE 2023, and at most 19% and 35%. Neural SDDP without iterations is 8 to 19 times more accurate in the median and 9 to 14 times in the worst case, and its abatement errors are 10 to 35 times smaller (Table 1, Figure 1). Its training is also cheaper: 90 seconds of SDDP for the training data and 11 minutes for the network, against 53 minutes for the deep equilibrium net (38 minutes for DICE 2023), all on four CPU cores. Section 7 discusses why this comparison favours neural SDDP.

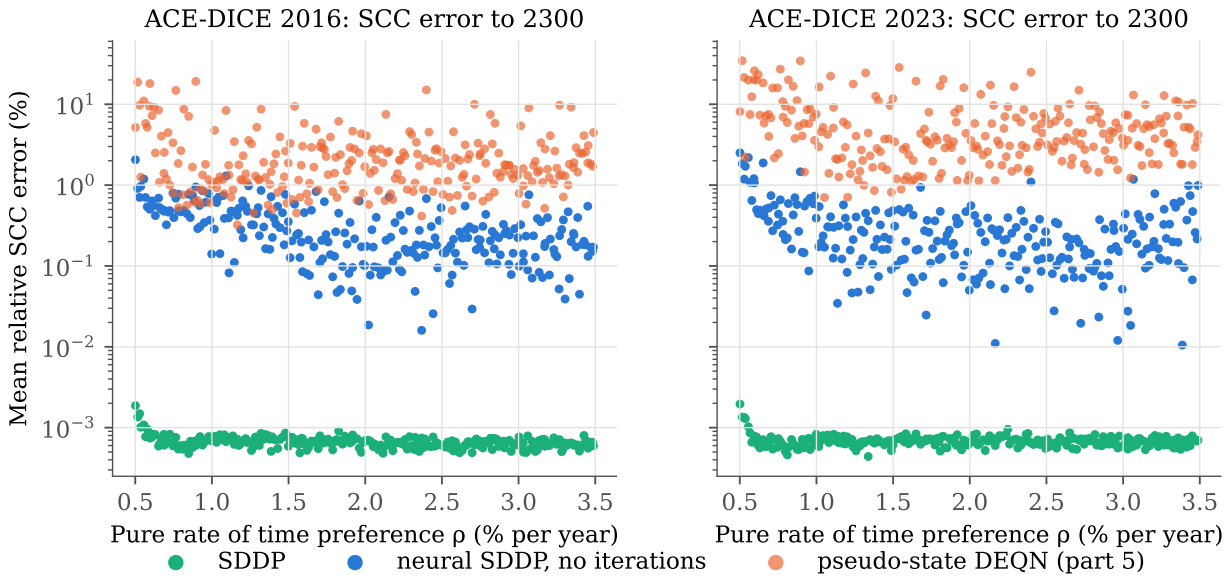


Figure 1: Mean relative SCC error over the evaluation window on the 256 test instances, against the time preference, for SDDP, neural SDDP without iterations, and the pseudo-state deep equilibrium net. Log scale.

6.4 Warm starts

With the predicted cuts, the first forward pass is already nearly optimal: its value is within 1.5×10^{-9} (DICE 2016) and 1.0×10^{-9} (DICE 2023) of the optimum, against 11.7% below it for SDDP from scratch, whose first forward pass is driven only by the provisional bound (Figure 4). But the predicted cuts are not guaranteed to lie above the value function, so they cannot certify convergence; a backward pass is needed either way. SDDP from scratch needs exactly one (Corollary 1), and both variants finish after two iterations, with the same 1,350 stage problems and the same accuracy (Table 2).

6.5 Productivity shocks

With i.i.d. shocks to log productivity with a standard deviation of 3%, SDDP runs six iterations of 2,700 stage problems each (16,200 in total, 31 seconds): forward passes along sampled shocks and backward passes over five Gauss–Hermite nodes. The cut slopes match the closed form to 10^{-14}, and along a sampled path, on which log capital deviates from the deterministic optimum by up to 0.07, the decisions match the deterministic optimal policy to within 3×10^{-5} for the consumption rate and 10^{-4} for the abatement rate. As ACE predicts, the shocks shift the value function but not its slopes, and with it, not the policy.

7 Discussion

Three routes to the same shadow price. In ACE, the SCC is a ratio of costates, and the three families of methods studied in this series reach the costates in different ways. Deep equilibrium nets

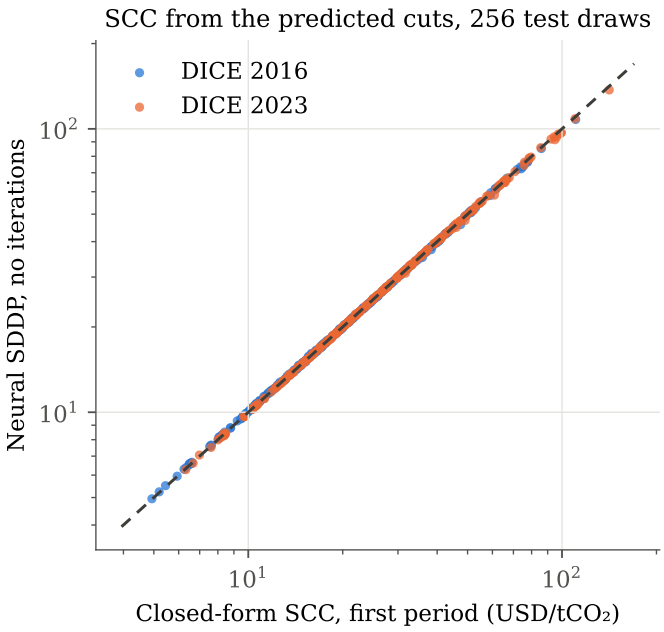


Figure 2: SCC in the first period from the predicted cuts, against the closed form, on the 256 test instances of both calibrations.

learn them as outputs that satisfy the residuals of the equilibrium conditions. Actor-critic methods obtain them as the gradient of a learned value function. SDDP computes them from the duals of convex stage problems, and Proposition 2 shows that this is the same costate recursion that the equilibrium conditions encode, evaluated exactly at a visited state instead of enforced approximately over a distribution of states. Seen this way, SDDP is a form of approximate dynamic programming in which the value function is represented by cuts and improved with exact dual information; its accuracy in ACE comes from the fact that this representation contains the true value function.

Why ACE is easy for SDDP. SDDP suits ACE unusually well: two decisions per period, convex stage problems, and an affine value function that a single cut represents exactly. Corollary 1 turns this into convergence after one backward pass. This is a strength of the benchmark, since every result can be verified, but it also means that ACE does not test what makes SDDP hard in general: value functions that need many cuts, high-dimensional decisions, and many scenarios per stage. The two-iteration convergence and the absent warm-start gains should be read in that light. They say that neural SDDP cannot improve on an exact method that is already fast, not that warm starts are useless.

What the comparison with the deep equilibrium net shows. Neural SDDP without iterations is about an order of magnitude more accurate than the pseudo-state deep equilibrium net on the same instances, at a quarter to a third of the training time. The comparison favours neural SDDP in two ways. First, its network is trained by supervised learning on exact solutions, which exist only because SDDP can solve every training instance; the deep equilibrium net learns from the equilibrium conditions alone and needs no solver. Second, the cut network has an easy target: a smooth map from three parameters and the period to six slopes and an intercept, the same at every state. The deep equilibrium net must represent policies and costates as functions of the

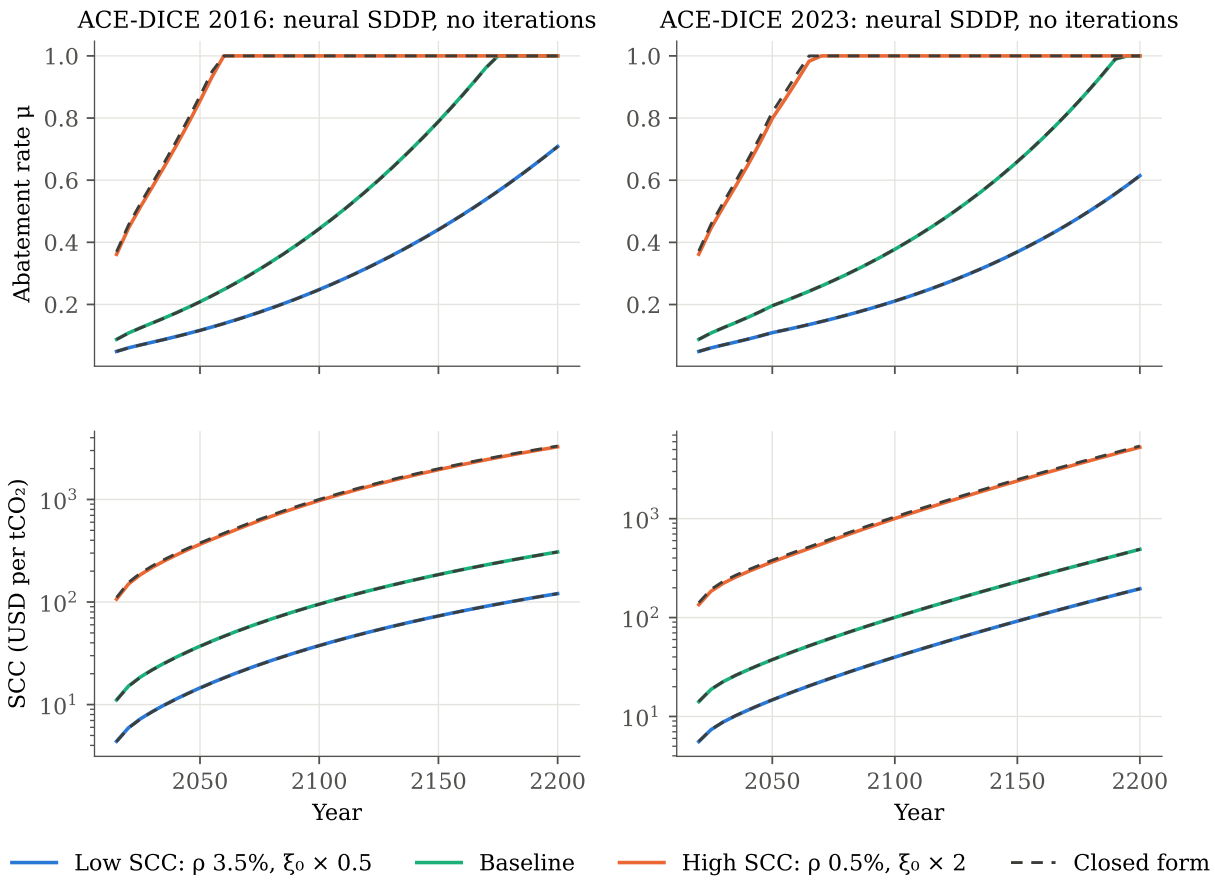


Figure 3: Abatement rate and SCC from neural SDDP without iterations (solid) and the closed form (dashed), for three instances: low SCC ($\rho = 3.5\%$, half the baseline damages), baseline, and high SCC ($\rho = 0.5\%$, twice the baseline damages).

full state, and its accuracy depends on where its simulated states fall. The fairer reading is that, when a convex solver can generate training data, learning the solution of the dual problem is much easier than learning the solution of the equilibrium conditions. Both approaches remain cheap to evaluate: the first-period SCC of a new instance needs only the predicted cut for period one and a single stage problem, so uncertainty quantification over many parameter draws is as feasible with neural SDDP as with the pseudo-state network.

Limits. The results rest on three assumptions. The first is convexity. The stage problems are convex only because business-as-usual emissions follow an exogenous path; if they are tied to capital, as in DICE, emissions become a product of the abatement rate and a convex function of capital, the value function need not be concave in the state, and cuts are no longer valid upper bounds. Tipping points with thresholds or regime switches (Lemoine and Traeger 2014; Cai and Lontzek 2019) break convexity in the same way. SDDP can still be run as a heuristic in such models, but without its bounds, and neural SDDP inherits that. The second is stagewise independence: SDDP handles i.i.d. shocks directly, but persistent shocks or learning about uncertain parameters require adding states, which in turn makes the value functions curved. The third is the finite horizon: as Remark 1 shows, carbon's shadow price converges only at the rate of discounting, so low discount

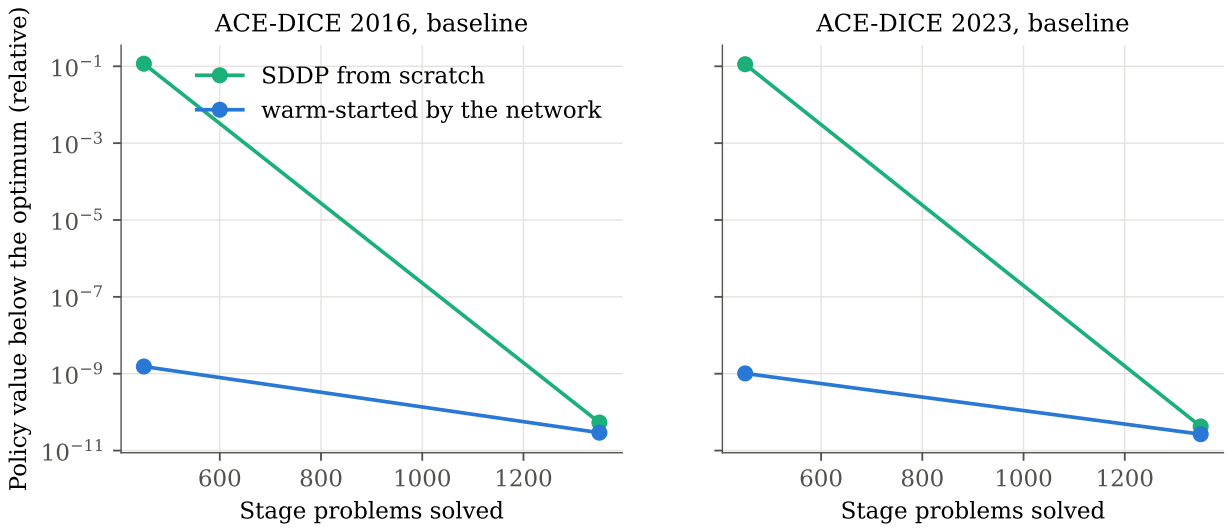


Figure 4: Relative shortfall of the forward-pass value from the optimum at baseline parameters, against the number of stage problems solved, for SDDP from scratch and warm-started with the predicted cuts. Log scale; each marker is one iteration.

rates require horizons of thousands of years, or a terminal value function. Finally, the cut network is trained with one seed per calibration, and I have not explored how many training instances are needed.

Where neural SDDP should pay off. The case for neural SDDP is strongest exactly where ACE stops being a good benchmark: models whose value functions are curved enough to need many cuts, so that SDDP needs many iterations and good initial cuts save most of them. A natural next step is a convex extension of ACE that keeps a closed form for part of the problem, for example stochastic damages with a convex hazard, followed by models without closed forms, for which the methods validated here would themselves become the benchmark.

8 Conclusion

With business-as-usual emissions taken as given, the Analytic Climate Economy decomposes into convex stage problems whose cut slopes are its costates. SDDP solves it exactly in two iterations, recovering the closed-form social cost of carbon to within 0.002% across a wide range of time preferences, damages and carbon-cycle parameters, and with productivity shocks it recovers the same costates and policy. Neural SDDP, trained on 128 SDDP solutions, predicts the cuts of new instances accurately enough that a single forward pass gives the SCC within 0.2% in the median, an order of magnitude more accurately than a pseudo-state deep equilibrium net. Its warm starts, the main source of speed-ups in the operations-research literature, do not help here, because the affine value function of ACE leaves SDDP nothing to gain. The lesson for climate economics is twofold: where a model is convex, cutting-plane methods offer exact, certifiable solutions and an easy target for learning; and the interesting tests for neural SDDP lie in the models, with curved value functions and uncertainty, that ACE was designed to avoid.

Code and Data Availability

All code, data and results are available at `https://github.com/hermanvest/DNN-for-ACE` in the directory `6_neural_sddp/`.

References

Agrawal, Akshay, Brandon Amos, Shane Barratt, Stephen Boyd, Steven Diamond, and J. Zico Kolter (2019). "Differentiable Convex Optimization Layers". In: *Advances in Neural Information Processing Systems (NeurIPS)*.

Azinovic, Marlon, Luca Gaegauf, and Simon Scheidegger (2022). "Deep Equilibrium Nets". In: *International Economic Review* 63.4, pp. 1471–1525.

Bae, Hyunglip, Jinkyu Lee, Woo Chang Kim, and Yongjae Lee (2023). "Deep Value Function Networks for Large-Scale Multistage Stochastic Programs". In: *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics (AISTATS)*. Vol. 206. Proceedings of Machine Learning Research.

Barrage, Lint and William Nordhaus (2023). *Policies, Projections, and the Social Cost of Carbon: Results from the DICE-2023 Model*. Working Paper w31112. National Bureau of Economic Research.

Benders, Jacques F. (1962). "Partitioning Procedures for Solving Mixed-Variables Programming Problems". In: *Numerische Mathematik* 4, pp. 238–252.

Cai, Yongyang and Thomas S. Lontzek (2019). "The Social Cost of Carbon with Economic and Climate Risks". In: *Journal of Political Economy* 127.6, pp. 2684–2734.

Dai, Hanjun, Yuan Xue, Zia Syed, Dale Schuurmans, and Bo Dai (2022). "Neural Stochastic Dual Dynamic Programming". In: *International Conference on Learning Representations (ICLR)*. arXiv:2112.00874.

Diamond, Steven and Stephen Boyd (2016). "CVXPY: A Python-Embedded Modeling Language for Convex Optimization". In: *Journal of Machine Learning Research* 17.83, pp. 1–5.

Dowson, Oscar and Lea Kapelevich (2021). "SDDP.jl: A Julia Package for Stochastic Dual Dynamic Programming". In: *INFORMS Journal on Computing* 33.1, pp. 27–33.

Folini, Doris, Aleksandra Friedl, Felix Kübler, and Simon Scheidegger (2024). "The Climate in Climate Economics". In: *The Review of Economic Studies*.

Friedl, Aleksandra, Felix Kübler, Simon Scheidegger, and Takafumi Usui (2023). "Deep Uncertainty Quantification: With an Application to Integrated Assessment Models". Working paper.

Girardeau, Pierre, Vincent Leclère, and Andrew B. Philpott (2015). "On the Convergence of Decomposition Methods for Multistage Stochastic Convex Programs". In: *Mathematics of Operations Research* 40.1, pp. 130–145.

Golosov, Mikhail, John Hassler, Per Krusell, and Aleh Tsyvinski (2014). "Optimal Taxes on Fossil Fuel in General Equilibrium". In: *Econometrica* 82.1, pp. 41–88.

Goulart, Paul J. and Yuwen Chen (2024). *Clarabel: An Interior-Point Solver for Conic Programs with Quadratic Objectives*. arXiv:2405.12762.

Kelley, James E. (1960). "The Cutting-Plane Method for Solving Convex Programs". In: *Journal of the Society for Industrial and Applied Mathematics* 8.4, pp. 703–712.

Kim, Chanyeong, Jongwoong Park, Hyunglip Bae, and Woo Chang Kim (2023). "Transformer-based Stagewise Decomposition for Large-Scale Multistage Stochastic Optimization". In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*.

Kingma, Diederik P. and Jimmy Ba (2017). *Adam: A Method for Stochastic Optimization.* arXiv:1412.6980.

Lee, Jinkyu, Sanghyeon Bae, Woo Chang Kim, and Yongjae Lee (2023). "Value Function Gradient Learning for Large-Scale Multistage Stochastic Programming Problems". In: *European Journal of Operational Research* 308.1, pp. 321–335.

Lemoine, Derek and Christian Traeger (2014). "Watch Your Step: Optimal Policy in a Tipping Climate". In: *American Economic Journal: Economic Policy* 6.1, pp. 137–166.

Maliar, Lilia, Serguei Maliar, and Pablo Winant (2021). "Deep Learning for Solving Dynamic Economic Models". In: *Journal of Monetary Economics* 122, pp. 76–101.

Nordhaus, William (2018). "Projections and Uncertainties about Climate Change in an Era of Minimal Climate Policies". In: *American Economic Journal: Economic Policy* 10.3, pp. 333–360.

Pereira, Mario V. F. and Leontina M. V. G. Pinto (1991). "Multi-Stage Stochastic Optimization Applied to Energy Planning". In: *Mathematical Programming* 52, pp. 359–375.

Shapiro, Alexander (2011). "Analysis of Stochastic Dual Dynamic Programming Method". In: *European Journal of Operational Research* 209.1, pp. 63–72.

Thieme, Herman Vestvik (2025). "Beyond the Curse of Dimensionality? Deep Equilibrium Nets for the Analytic Climate Economy (revised)". Master's thesis, revised; `https://github.com/hermanvest/DNN-for-ACE`.

Traeger, Christian P. (2021). "IAMs and CO2 Emissions—An Analytic Discussion". In: *EAERE Conference*.

— (2023). "ACE—Analytic Climate Economy". In: *American Economic Journal: Economic Policy* 15.3, pp. 372–406.